System Design and AI Integration

AI Role and Boundaries

The AI (Claude claude-sonnet-4-5) has a precisely scoped role in the pipeline: it generates two pieces of text per at-risk student - a two-sentence facilitator summary that contextualises the student's engagement pattern, and a WhatsApp message draft addressed to the parent. It does nothing else. Risk scoring is performed by a deterministic weighted formula, not the AI. The decision about which students are at-risk is made by that formula before the AI is ever called. The AI receives only students who have already been identified as CRITICAL or HIGH risk.

This boundary is intentional. AI-generated risk scores would be opaque and unjustifiable to a facilitator who asks "why is this student flagged?" A deterministic formula produces an auditable answer: "attendance_rate is 0.42, contributing 35 points to a risk score of 78." That level of explainability is essential for facilitator trust and for regulatory compliance if the pipeline is ever audited. The AI's strength is natural language generation - writing warm, contextually appropriate messages - not numerical scoring.

The claude-sonnet-4-5 model was selected for this role based on its balance of output quality and API cost. It produces grammatically correct Arabic-compatible messages and coherent facilitator summaries without requiring the latency or cost of a larger frontier model. Tool use (structured JSON output) is used rather than free-form generation, which eliminates the need for brittle response parsing and makes the AI's output directly machine-readable.

What AI Does Not Do

The AI does not decide which students to contact. That decision is made by the risk formula and the CRITICAL/HIGH threshold, both of which are set by human- defined constants that can be tuned by the academic director. The AI is not in the decision loop; it is in the communication drafting loop. A student's inclusion in the intervention list is deterministic and auditable without any AI involvement.

The AI does not send messages. The whatsapp_messages.csv is a draft document reviewed by the facilitator before any message is sent. The pipeline has no integration with WhatsApp Business API, no email server, and no notification system. Every message leaves the pipeline as a text string in a CSV cell; it becomes a sent message only after a human facilitator copies it into WhatsApp and presses send. This human-in-the-loop requirement is a design choice, not a limitation to be removed in future iterations.

The AI does not access real student systems, school databases, or any external data source other than what is provided in the cohort prompt. Its context is limited to the engagement metrics included in the API call. It does not know a student's grades, attendance history beyond what the CSV contains, or any information from other systems. This containment is both a privacy protection and a scope boundary: the AI can only say things that are grounded in the data the pipeline has ingested.

Accuracy vs Cost vs Latency Tradeoffs

The three primary AI integration tradeoffs - accuracy, cost, and latency - were resolved in favour of cost and latency for this use case. The pipeline runs once daily; a 60-second total runtime including 14 API calls is entirely acceptable. There is no real-time requirement that would justify paying for a faster but more expensive model tier or a dedicated API capacity reservation.

Campus batching (one API call per campus rather than per student) is the primary cost optimisation. In the 20-campus demo run, it reduced API calls from approximately 120 to 14 - a 9x reduction. The cost difference at claude-sonnet-4-5 pricing is material: per-student calls would have used roughly 9x more tokens (approximately 285,939 tokens vs 31,771 actual). At scale, campus batching is what makes the $200/month budget feasible.

Message accuracy is acceptable rather than optimal. The AI is given only aggregate metrics (attendance rate, practice question count, trend direction, days since last note) per student, not the full note text or historical detail. This keeps prompts short and costs low, but it means the AI cannot reference specific events or conversations. The facilitator summary and WhatsApp message are therefore general rather than highly personalised. For the target use case - a facilitator who may be managing 15 students and needs a starting point for each conversation - this level of accuracy is fit for purpose.

Human Review Loop

The pipeline is designed around a mandatory human review step. The facilitator reviews intervention_priority_list.xlsx before contacting any family. The Excel file presents students in CRITICAL-first order with all supporting metrics visible, enabling the facilitator to make informed judgements about which students to prioritise and whether the risk assessment aligns with their direct knowledge of the student.

WhatsApp message drafts in the per-campus dashboard files are explicitly labelled as drafts. The pipeline does not prevent a facilitator from sending a template message verbatim, but the expectation is that the facilitator will read the message, adjust the tone for the specific family relationship, and send it through their personal WhatsApp application. The two-step process (AI drafts, human reviews, human sends) is the core of the intervention workflow and is not intended to be automated away.

The generated_by column in every campus dashboard file makes the distinction between AI-generated and template-generated messages explicit. A facilitator who sees generated_by: template knows that the API was not available for that student's message and should apply extra judgement when reviewing it. This transparency about the message provenance is part of the human review loop design - facilitators should never be uncertain about whether they are reading AI output or rule-based output.

Failure Modes

Three primary failure modes were identified during design and addressed explicitly. The first is API unavailability: if the Anthropic API is down or rate-limiting, the three-layer fallback (SDK retry

-> re-prompt -> template) ensures the pipeline completes with template-generated messages rather than crashing. The output is degraded but complete; the facilitator still receives a usable intervention list.

The second failure mode is malformed or missing input CSV data. ingestion.py handles this through type coercion with pd.to_numeric(errors='coerce'), deduplication, and conservative missing-value fill. No input anomaly - missing rows, type mismatch strings, duplicate IDs - causes the pipeline to crash. All anomalies are logged at WARNING level and the pipeline continues. A student with completely missing metrics data receives a maximum risk score (conservative default), ensuring they are not invisibly excluded from the intervention list.

The third failure mode is an empty campus (a campus_id present in the metadata with no CRITICAL or HIGH students). The LLM batching loop skips campuses with no at-risk students without error; the per-campus dashboard is still written (with all students at MEDIUM or LOW) but no API call is made. This is the expected happy-path outcome for a campus that is performing well, and it should not produce any log warnings - only an INFO-level note that the campus was processed with zero at-risk students.